

Nanyang Technological University

Domain Expansion: Infinite Penalty(ˆ. .ˆ) ∫

Peeranat Kongkijpipat (autoastt), Nathan Juirnarongrit (saltyy), Kittiphat Pongarunotai (Marszpace)

2026-11-11

Contents

1 Contest 1
2 Mathematics 1
3 Data structures 1
4 Numerical 4
5 Number theory 7
6 Combinatorial 10
7 Graph 11
8 Geometry 17
9 Strings 20
10 Various 22

Contest (1)

macros.cpp 7 lines

```
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define pb push_back
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
```

hash.sh 7 lines

```
#!/bin/bash
# Prints the first 6 chars of the md5 of a file (or stdin),
# ignoring whitespace and comments: to check typed code.
# Usage: chmod +x hash.sh; ./hash.sh [file.cpp]
[ -z "$1" -o -r "$1" ] || { echo "can't read $1"; exit 1; }
cpp -dD -P -fpreprocessed < "${1:-/dev/stdin}" |
  tr -d '[:space:]' | md5sum | cut -c-6
```

stress.sh 15 lines

```
# Stress test. Compile like the judge, plus -Wall and
# -D_GLIBCXX_ASSERTIONS (vector/string out-of-range aborts).
# ./A solution, ./gen S prints a small random test (seed S),
# ./A2 brute force (may be missing if ./checker exists),
# ./checker in out ans: optional, exit 0 = OK (ans = A2 output)
# Usage: bash stress.sh [iterations]
for ((i=1; i<=${1:-1000}; i++)); do
  ./gen $i > in
  timeout 5 ./A < in > out ||
    { echo "A crash/TLE, seed $i"; exit 1; }
  if [ -x A2 ]; then ./A2 < in > ans; else : > ans; fi
  if [ -x checker ]; then ./checker in out ans
  else diff -wB out ans >/dev/null; fi || {
    echo "WA, seed $i"; tail -n+1 in out ans; exit 1; }
done; echo OK
```

interactive.sh 24 lines

```
# Interactive tester. ./A solution; ./interactor in reads the
# test file "in", talks on stdin/stdout (flush every line!),
# verdict on stderr, exit != 0 on WA; ./gen S prints a test.
# Usage: bash interactive.sh in        one test
#        bash interactive.sh gen [N]   stress test
run() {
  timeout 5 ./interactor $1 > to < from 2> verdict & p=$!
  timeout 5 ./A < to > from; ar=$?  # keep this open order
# Transcript in "log": use these lines, not the 2 above
# : > log
# timeout 5 ./interactor $1 > >(tee -a log > to) < from \
#   2> verdict & p=$!
# timeout 5 ./A < to > >(tee -a log > from); ar=$?
  wait $p; ir=$?
  [ $ar -eq 0 -a $ir -eq 0 ] && return
  echo "A exit $ar (124 = timeout: flush?)," \
    "judge exit $ir: $(cat verdict)"; return 1
}
mkfifo to from; trap 'rm -f to from' EXIT
if [ "$1" = gen ]; then
  for ((i=1; i<=${2:-100}; i++)); do ./gen $i > in
    run in || { echo "seed $i"; exit 1; }
  done; echo OK
else run $1 && cat verdict; fi
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v\cos w + \cos v\sin w$$
$$\cos(v+w) = \cos v\cos w - \sin v\sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v\tan w}$$
$$\sin v + \sin w = 2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v + \cos w = 2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$(V+W)\tan(v-w)/2 = (V-W)\tan(v+w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \text{atan2}(b, a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.2.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180°, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.2.3 Lattice polygons

Pick's theorem: a simple polygon with integer vertices has area $A = I + B/2 - 1$, where I, B count the interior and boundary lattice points. The segment from (x_1, y_1) to (x_2, y_2) contains $\gcd(|x_2 - x_1|, |y_2 - y_1|) + 1$ lattice points.

2.3 Useful Formulae

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\text{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$
$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$
$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

Data structures (3)

OrderStatisticTree.h 8f2f6d, 19 lines

Description: Set (not multiset) with `find_by_order` and `order_of_key`. For a map, change `null_type`. **Time:** $\mathcal{O}(\log N)$

```
#pragma push_macro("int")
#undef int
#include <bits/extc++.h>
#pragma pop_macro("int")
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void example() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

8a5c0c, 10 lines

Description: unordered_map API, about 3x faster, 1.5x memory. Initial capacity must be a power of 2.

```
#pragma push_macro("int")
#undef int
#include <bits/extc++.h>
#pragma pop_macro("int")
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

SegmentTree.h

0f4bdb, 19 lines

Description: 0-indexed point update. Max by default: change T, f, unit. **Time:** $\mathcal{O}(\log N)$

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegTreeBlock.h

d22e37, 18 lines

Description: The part of LazySeg you edit. Default: range chmin and add; sum / max / count of max (beats). chmin $v = \{$v, 0$\}$, add $v = \{$INF, v$\}$; S() is a leaf with value 0. comp must keep tags closed, app must act on the aggregate as the tag acts on each element. **Add only:** ok $= 1$, drop se, cnt. **Min query:** add mn, op takes min, app: mn = min(mn, x.c) + x.a. **Chmax:** store $-a_i$, negate updates and answers. **Set, add, mul:** L $= (a, b)$: $x \mapsto ax + b$, id $= \{1, 0\}$, set $v = \{0, v\}$, add $= \{1, v\}$, mul $= \{v, 0\}$, app: a*s + b*len (sum) or a*s + b (max, $a \geq 0$), comp(x, y) $= \{$x.a*y.a, x.a*y.b + x.b$\}$, ok $= 1$. **Chmin, chmax, add, set; max / min only:** L $= (a, lo, hi)$: $x \mapsto \min(\max(x + a, lo), hi)$, set $v = \{0, v, v\}$, comp(x, y): $a =$ x.a + y.a, lo, hi = y.lo, y.hi plus x.a, clamped to [x.lo, x.hi].

```
const ll INF = 1e18;
struct S { ll mx = 0, se = -INF, cnt = 1, sum = 0; };
struct L { ll c, a; }; // tag x -> min(x, c) + a
S e() { return {-INF, -INF, 0, 0}; }
L id() { return {INF, 0}; }
S op(S a, S b) {
  if (a.mx < b.mx) swap(a, b);
  if (a.mx == b.mx) return {a.mx, max(a.se, b.se), a.cnt + b.
      cnt, a.sum + b.sum};
  return {a.mx, max(a.se, b.mx), a.cnt, a.sum + b.sum};
}
bool ok(S s, L x) { return x.c > s.se; } // x on all of s?
S app(S s, L x, int len) {
  if (x.c < s.mx) s.sum -= (s.mx - x.c) * s.cnt, s.mx = x.c;
  s.sum += x.a * len; s.mx += x.a;
  if (s.se > -INF / 2) s.se += x.a;
  return s;
}
L comp(L x, L y) { return {min(y.c, x.c - y.a), x.a + y.a}; }
```

LazySegTree.h

f8c112, 42 lines

Description: Lazy segment tree; edit only LazySegTreeBlock.h. app(s, x, len) applies tag x to a segment of length len; comp(x, y) $= y$ first, then x. Ranges $[l, r)$. maxRight(l, f): largest r with f(query(l, r)), for monotone f with f(e()) true. **Time:** $\mathcal{O}(\log N)$

Usage: vector<S> v(n); LazySeg t(v); t.upd(l, r, {5, 0}); t.query(l, r).sum;
t.maxRight(l, [&](S s) { return s.mx < x; }); // first index >= l with value >= x

```
struct LazySeg {
  int n; vector<S> t; vector<L> z;
  LazySeg(vector<S> v) : n(sz(v)), t(4*n), z(4*n, id()) { build
      (v, 1, 0, n); }
  void build(vector<S>& v, int o, int lo, int hi) {
    if (hi - lo == 1) { t[o] = v[lo]; return; }
    int m = (lo + hi) / 2;
    build(v, 2*o, lo, m); build(v, 2*o+1, m, hi);
    t[o] = op(t[2*o], t[2*o+1]);
  }
  void ap(int o, int len, L x) { t[o] = app(t[o], x, len); z[o]
       = comp(x, z[o]); }
  void push(int o, int lo, int hi) {
    int m = (lo + hi) / 2;
    ap(2*o, m-lo, z[o]); ap(2*o+1, hi-m, z[o]); z[o] = id();
  }
  void upd(int l, int r, L x, int o=1, int lo=0, int hi=-1) {
    if (hi < 0) hi = n;
    if (r <= lo || hi <= l) return;
    if (l<=lo && hi<=r && ok(t[o], x)) return ap(o, hi-lo, x);
    push(o, lo, hi); int m = (lo + hi) / 2;
    upd(l, r, x, 2*o, lo, m); upd(l, r, x, 2*o+1, m, hi);
    t[o] = op(t[2*o], t[2*o+1]);
  }
  S query(int l, int r, int o = 1, int lo = 0, int hi = -1) {
    if (hi < 0) hi = n;
    if (r <= lo || hi <= l) return e();
    if (l <= lo && hi <= r) return t[o];
    push(o, lo, hi); int m = (lo + hi) / 2;
    return op(query(l,r,2*o,lo,m), query(l,r,2*o+1,m,hi));
  }
  template<class F> int maxRight(int l, F f) {
    S a = e();
    auto go = [&](auto&& go, int o, int lo, int hi) -> int {
      if (hi <= l) return hi;
      if (l<=lo && f(op(a, t[o]))) return a = op(a, t[o]), hi;
      if (hi - lo == 1) return lo;
      push(o, lo, hi); int m = (lo + hi) / 2;
      int r = go(go, 2*o, lo, m);
      return r < m ? r : go(go, 2*o+1, m, hi);
    };
    return go(go, 1, 0, n);
  }
};
```

PersistentSegmentTree.h

7845a1, 31 lines

Description: Sums over positions $[0, n)$. upd adds x at p and returns the new root; query(v, l, r) sums $[l, r)$. **Time:** $\mathcal{O}(\log N)$ per operation, $\mathcal{O}(\log N)$ memory per update

Usage: PST t(n); r[i+1] = t.upd(r[i], p, 1); t.kth(r[l], r[r], k);

```
struct PST { // version = root, 0 = empty
  struct Nd { int l = 0, r = 0; ll s = 0; };
  vector<Nd> t{Nd()}; int n;
  PST(int n) : n(n) {}
  int upd(int v, int p, ll x, int lo = 0, int hi = -1) {
    if (hi < 0) hi = n;
    int c = sz(t), m = (lo + hi) / 2, ch;
    t.push_back(t[v]); t[c].s += x;
    if (hi - lo > 1) {
      if (p < m) ch = upd(t[v].l, p, x, lo, m), t[c].l = ch;
      else ch = upd(t[v].r, p, x, m, hi), t[c].r = ch;
    }
    return c;
  }
  ll query(int v, int l, int r, int lo = 0, int hi = -1) {
    if (hi < 0) hi = n;
    if (!v || r <= lo || hi <= l) return 0;
    if (l <= lo && hi <= r) return t[v].s;
    int m = (lo + hi) / 2;
    return query(t[v].l,l,r,lo,m) + query(t[v].r,l,r,m,hi);
  }
  int kth(int a, int b, ll k) { // k-th (0-idx) in b - a
    int lo = 0, hi = n;
    while (hi - lo > 1) {
      int m = (lo + hi) / 2; ll c = t[t[b].l].s - t[t[a].l].s;
      if (k < c) a = t[a].l, b = t[b].l, hi = m;
      else k -= c, a = t[a].r, b = t[b].r, lo = m;
    }
    return lo;
  }
};
```

UnionFindRollback.h

a88960, 23 lines

Description: DSU with undo (rollback(time())); skip st, time, rollback if not needed. If join returns false and par(a) $\oplus$ par(b) $\neq w$, the demand is contradictory. **Time:** $\mathcal{O}(\log(N))$

Usage: int t = uf.time(); ...; uf.rollback(t);

```
struct RollbackUF {
  vi e, d; vector<pii> st;
  RollbackUF(int n) : e(n, -1), d(n) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int par(int x) { return e[x] < 0 ? 0 : d[x] ^ par(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b, int w = 0) { // par(a)^par(b) = w
    int x = par(a) ^ par(b) ^ w;
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a; d[b] = x;
    return true;
  }
};
```

DynCon.h

"UnionFindRollback.h" 53ca31, 24 lines

Description: Offline dynamic connectivity. add(l, r, a, b): edge a–b during times $[l, r)$. run(f) calls f(t) for $t = 0..T-1$ with uf $=$ the graph at time t. Toggles: map of start times. **Time:** $\mathcal{O}((N + E \log T) \log N)$

Usage: DynCon dc(n, T); dc.add(l, r, a, b);
dc.run([&](int t){ ... dc.uf.find(x) ... });

```
struct DynCon { // bad: odd-cycle edges, 0 = bipartite
  int T, bad = 0; RollbackUF uf; vector<vector<pii>> seg;
  DynCon(int n, int T) : T(T), uf(n), seg(4*T + 4) {}
  void add(int l, int r, int a, int b, int o = 1, int lo = 0,
       int hi = -1) {
```

```
    if (hi < 0) hi = T;
    if (r <= lo || hi <= l) return;
    if (l<=lo && hi<=r) { seg[o].push_back({a, b}); return; }
    int m = (lo + hi) / 2;
    add(l, r, a, b, 2*o, lo, m); add(l, r, a, b, 2*o+1, m, hi);
  }
  template<class F> void run(F&& f, int o = 1, int lo = 0, int
      hi = -1) {
    if (hi < 0) hi = T;
    if (lo >= hi) return;
    int t = uf.time(), b0 = bad;
    for (auto [a, b] : seg[o])
      if (!uf.join(a, b, 1) && uf.par(a) == uf.par(b)) bad++;
    if (hi - lo == 1) f(lo);
    else {
      int m = (lo + hi) / 2;
      run(f, 2*o, lo, m); run(f, 2*o+1, m, hi);
    }
    uf.rollback(t); bad = b0;
  }
};
```

Matrix.h

4da5a2, 26 lines

Description: Square matrices with $\star$ and power ^.
Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

LineContainer.h

8ec1c7, 30 lines

Description: Add lines $kx+m$, query the maximum at x (convex hull trick).
Time: $\mathcal{O}(\log N)$

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

LiChao.h

8f3109, 30 lines

Description: Minimum of lines $kx+m$ at integer $x \in [0, n)$; `addSeg` only on $x \in [a, b)$. Max: negate k, m and the answer. **Time:** $\mathcal{O}(\log N)$ per operation ($\mathcal{O}(\log^2 N)$ for `addSeg`)

```
struct LiChao {
  static const ll INF = 4e18;
  struct Ln { ll k=0, m=INF; ll at(ll x) { return k*x + m; } };
  int n; vector<Ln> t;
  LiChao(int n) : n(n), t(4*n) {}
  void ins(Ln ln, int o, int lo, int hi) {
    int mid = (lo + hi) / 2;
    bool l=ln.at(lo)<t[o].at(lo), m=ln.at(mid)<t[o].at(mid);
    if (m) swap(t[o], ln);
    if (hi - lo == 1) return;
    if (l != m) ins(ln,2*o,lo,mid); else ins(ln,2*o+1,mid,hi);
  }
  void addSeg(ll k, ll m, int a, int b, int o = 1, int lo = 0,
      int hi = -1) {
    if (hi < 0) hi = n;
    if (b <= lo || hi <= a) return;
    if (a <= lo && hi <= b) return ins({k, m}, o, lo, hi);
    int mid = (lo + hi) / 2;
    addSeg(k,m,a,b,2*o,lo,mid); addSeg(k,m,a,b,2*o+1,mid,hi);
  }
  void add(ll k, ll m) { addSeg(k, m, 0, n); }
  ll query(int x) { // INF if no line covers x
    ll r = INF;
    for (int o = 1, lo = 0, hi = n; ; ) {
      r = min(r, t[o].at(x));
      if (hi - lo == 1) return r;
      int mid = (lo + hi) / 2;
      if (x < mid) o = 2*o, hi = mid; else o = 2*o+1, lo = mid;
    }
  }
};
```

Treap.h

08bcd4, 82 lines

Description: Implicit treap (a sequence, split / merge by position) keeping subtree sums with lazy range add and reverse. `go(t, l, r, f)` runs `f` on the node holding positions $[l, r)$. **Time:** $\mathcal{O}(\log N)$

```
struct Node {
  Node *l = 0, *r = 0;
  ll val, sum, add = 0;
  int y, c = 1;
  bool rev = 0;
  Node(ll val) : val(val), sum(val), y(rand()) {}
  void recalc(), push();
  void app(ll a, bool f) { // add a to all values, reverse if f
    val += a, sum += a * c, add += a;
    if (f) rev ^= 1, swap(l, r);
  }
};

int cnt(Node* n) { return n ? n->c : 0; }
ll sm(Node* n) { return n ? n->sum : 0; }
void Node::recalc() {
  c = cnt(l) + cnt(r) + 1, sum = sm(l) + sm(r) + val;
}
void Node::push() {
  if (l) l->app(add, rev);
  if (r) r->app(add, rev);
  add = rev = 0;
}

template<class F> void each(Node* n, F f) {
  if (n) {
    n->push(); each(n->l, f); f(n->val); each(n->r, f);
  }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  n->push();
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto [L,R] = split(n->l, k);
    n->l = R;
    n->recalc();
    return {L, n};
  } else {
    auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and just "k"
    n->r = L;
    n->recalc();
    return {n, R};
  }
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->push();
    l->r = merge(l->r, r);
    return l->recalc(), l;
  } else {
    r->push();
    r->l = merge(l, r->l);
    return r->recalc(), r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l,r] = split(t, pos);
  return merge(merge(l, n), r);
}

// Run f on the subtree of positions [l, r), e.g.
// go(t, l, r, [&](Node* n) { n->app(5, 0); }); // add 5
// app(0, 1) reverses, n->sum is the sum (f skipped if empty).
template<class F> void go(Node*& t, int l, int r, F f) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
  if (b) f(b);
  t = merge(merge(a, b), c);
}
```

```
// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}
```

FenwickTree.h

e62fac, 22 lines

Description: 0-indexed. `lower_bound` needs non-negative values. **Time:** Both operations are $\mathcal{O}(\log N)$.

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

"FenwickTree.h" 157f07, 22 lines

Description: 0-indexed. Points to update must be known: `fakeUpdate` each, then `init()`. **Time:** $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) { // sum over [0, x) x [0, y)
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

RMQ.h

510c32, 16 lines

Description: Sparse table (min). **Time:** $\mathcal{O}(|V| \log |V| + Q)$
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`

```
template<class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b) { // min of V[a..b), O(1)
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

MoQueries.h

a12ef4, 49 lines

Description: Mo's algorithm on intervals (`mo`) or tree paths (`moTree`); fill in `add`, `del`, `calc`. Values on tree edges: `step` toggles the edge (a, c) and the initial `add` goes (keep `in`). **Time:** $\mathcal{O}\left(N\sqrt{Q}\right)$

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) { // queries [l, r)
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  }
};
```

PolyInterpolate.h

08bf48, 13 lines

Description: Coefficients a of the degree $n-1$ polynomial through (x_i, y_i). Precision: pick $x_k = c\cos(k\pi/(n-1))$. **Time:** $\mathcal{O}\left(n^2\right)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) { // p(x) = sum a_i x^i
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

LagrangeInterpolation.h

"../number-theory/ModPow.h" 4f0513, 18 lines

Description: $f(x) \bmod p$ from $y_i = f(i)$, $i < n$, $\deg f < n$, p prime $> n$. Sums of powers: $\sum_{i \le x} i^k$ has degree $k+1$, so give it $k+2$ prefix sums. **Time:** $\mathcal{O}(n)$

```
ll lagrange(const vector<ll>& y, ll x) {
  int n = sz(y);
  x = (x % mod + mod) % mod;
  if (x < n) return y[x];
  vector<ll> pre(n + 1, 1), suf(n + 1, 1), ifact(n, 1);
  rep(i,0,n) pre[i+1] = pre[i] * ((x - i + mod) % mod) % mod;
  for (int i = n; i--;) suf[i] = suf[i+1] * ((x - i + mod) %
       mod) % mod;
  ll f = 1;
  rep(i,1,n) f = f * i % mod;
  ifact[n-1] = modpow(f, mod - 2);
  for (int i = n-1; i--;) ifact[i] = ifact[i+1] * (i+1) % mod;
  ll ans = 0;
  rep(i,0,n) {
    ll t = y[i] * pre[i] % mod * suf[i+1] % mod * ifact[i] %
         mod * ifact[n-1-i] % mod;
    ans = (ans + ((n - 1 - i) & 1 ? mod - t : t)) % mod;
  }
  return ans;
```

```
}
```

BerlekampMassey.h

"../number-theory/ModPow.h" 96548b, 20 lines

Description: Shortest linear recurrence (in linearRec's format) of a sequence mod prime, from its first $2n$ terms. For guessing recurrences. **Time:** $\mathcal{O}(N^2)$

Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

"../number-theory/ModPow.h" f4e444, 26 lines

Description: $S[i] = \sum_j S[i-j-1]\,tr[j]$, given $S[0..n-1]$ and $tr[0..n-1]$, mod mod. **Time:** $\mathcal{O}(n^2 \log k)$

Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) { // k-th term, 0-indexed
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

BostanMori.h

"NumberTheoreticTransform.h" 75eebc, 16 lines

Description: Same as linearRec mod 998244353 (values in $[0, mod)$), $O(n \log n \log k)$: halves k via $P(x)Q(-x)/Q(x)Q(-x)$. **Time:** $\mathcal{O}(n \log n \log k)$

Usage: bostanMori({0, 1}, {1, 1}, k) // k'th Fibonacci number

```
ll bostanMori(const vl& S, const vl& tr, ll k) {
  int n = sz(tr);
  if (!n) return 0;
  vl Q(n + 1, 1);
  rep(j,0,n) Q[j + 1] = (mod - tr[j]) % mod;
  vl P = conv(S, Q);
  P.resize(n);
  for (; k; k /= 2) {
    vl R = Q;
    for (int i = 1; i <= n; i += 2) R[i] = (mod - R[i]) % mod;
    vl U = conv(P, R), V = conv(Q, R);
    rep(i,0,n) P[i] = U[2 * i + k % 2];
    rep(i,0,n+1) Q[i] = V[2 * i];
  }
  return P[0];
}
```

4.2 Optimization

HillClimbing.h

8eeeaf, 14 lines

Description: Minimizes f over the plane from start; fine for unimodal f.

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
  pair<double, P> cur(f(start), start);
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.second;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
    }
  }
  return cur;
}
```

Integrate.h

4756fc, 7 lines

Description: Simpson's rule with $2n$ intervals; error $\propto h^4$, check that the result is stable when n changes.

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}
```

4.3 Matrices

Determinant.h

bd5cec, 15 lines

Description: Doubles. Destroys the matrix. **Time:** $\mathcal{O}(N^3)$

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

"../number-theory/ModPow.h" 5e85f0, 17 lines

Description: Determinant mod any mod (also composite) by Euclid row steps. Remove the % mod for exact integers. **Time:** $\mathcal{O}(N^3)$

```
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

44c9ab, 38 lines

Description: Solves $Ax = b$. Returns rank, or -1 if no solution (an arbitrary one if rank $< m$). Destroys A, b. **Time:** $\mathcal{O}(n^2 m)$

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

"SolveLinear.h" 08e495, 7 lines

Description: Changes to SolveLinear to get back all uniquely determined values of x:

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
```

```
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

fa2d7a, 34 lines

Description: Over $\mathbb{F}_2$. Returns rank, or -1 if none. Destroys A, b. **Time:** $\mathcal{O}(n^2 m)$

```
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinearMod.h

"../number-theory/ModPow.h" 5008e8, 29 lines

Description: Mod prime, entries in $[0, mod)$. Returns rank, or -1 if none; free variables are 0. Destroys A, b. **Time:** $\mathcal{O}(nm \min(n, m))$

```
typedef vector<ll> vl;
int solveLinearMod(vector<vl>& A, vl& b, vl& x) {
  int n = sz(A), m = sz(x), rank = 0;
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    int br = -1, bc = 0;
    rep(r,i,n) rep(c,i,m) if (br<0 && A[r][c]) br = r, bc = c;
    if (br < 0) {
      rep(j,i,n) if (b[j]) return -1;
      break;
    }
    swap(A[i], A[br]); swap(b[i], b[br]); swap(col[i],col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    ll iv = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll fac = A[j][i] * iv % mod;
      b[j] = (b[j] - fac * b[i] % mod + mod) % mod;
      rep(k,i+1,m) A[j][k] = (A[j][k] - fac * A[i][k] % mod +
          mod) % mod;
    }
    rank++;
  }
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] = b[i] * modpow(A[i][i], mod - 2) % mod;
    x[col[i]] = b[i];
    rep(j,0,i) b[j] = (b[j] - A[j][i]*b[i] % mod + mod) % mod;
  }
  return rank; // (multiple solutions if rank < m)
}
```

XorBasis.h

89ae6a, 28 lines

Description: Basis of the span under xor. `insert` returns whether x was independent; `has(x)`: x in the span. **Time:** $\mathcal{O}(64)$ per operation

```
typedef unsigned long long ull;
struct XorBasis {
  ull b[64] = {}; int rk = 0;
  bool insert(ull x) {
    for (int i = 64; i--;) if (x >> i & 1) {
      if (!b[i]) { b[i] = x; rk++; return 1; }
      x ^= b[i];
    }
    return 0;
  }
  bool has(ull x) {
    for (int i = 64; i--;) if (x >> i & 1) {
      if (!b[i]) return 0;
      x ^= b[i];
    }
    return 1;
  }
  ull best(ull x = 0) { // max x ^ s over the span
    for (int i = 64; i--;) x = max(x, x ^ b[i]);
    return x;
  }
  ull kth(ull k) { // k-th smallest, 0-idx, k < 2^rk
    rep(i,0,64) rep(j,0,i) if (b[j] && (b[i] >> j & 1)) b[i] ^=
        b[j];
    ull r = 0; int t = 0;
    rep(i,0,64) if (b[i] && (k >> t++ & 1)) r ^= b[i];
    return r;
  }
};
```

MatrixInverse.h

ebfff6, 35 lines

Description: Returns rank; A^{-1} is stored in A if rank $= n$. Mod p^k: start from the inverse mod p and repeat $A^{-1} \leftarrow A^{-1}(2I - AA^{-1})$, doubling k each time. **Time:** $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

4.4 Fourier transforms

FastFourierTransform.h

00ced6, 35 lines

Description: Convolution with doubles. Exact while $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}), else NTT / FFTMod. By hand: FFT, multiply, divide by n, reverse(start+1, end), FFT back. **Time:** $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) { // c[x] = sum a[i]b[x-i]
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

"FastFourierTransform.h" 2bc998, 22 lines

Description: Convolution modulo any `M`, inputs in $[0, M)$, while $N \log_2 N \cdot M < 8.6 \cdot 10^{14}$ (in practice 10^{16}). **Time:** $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
```

```
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=(int)sqrt(M);
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

"../number-theory/ModPow.h"

a92f24, 36 lines

Description: Convolution mod `mod`, a prime $c \cdot 2^k + 1$ such as 998244353, result size $\leq 2^k$ (any modulus: FFTMod). Inputs in $[0, mod)$. By hand: NTT, multiply, divide by n, reverse(start+1, end), NTT back. **Time:** $\mathcal{O}(N \log N)$

```
// mod = c * 2^k + 1: 998244353 = (119 << 23) + 1. Same root for
// 5 << 25, 7 << 26, 479 << 21 and 483 << 21 (each + 1).
static_assert((mod - 1) % (1 << 21) == 0, "mod: NTT prime");
const ll root = 62;
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) { // c[x] = sum a[i]b[x-i]
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

464cf3, 16 lines

Description: AND / OR / XOR convolution $c[z] = \sum_{x \oplus y = z} a[x]b[y]$ (choose the line in `FST`); size a power of 2. **Time:** $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

PolyOps.h

"NumberTheoreticTransform.h"

fe06e0, 68 lines

Description: Power series mod x^n and `mod` (NTT prime); results have exactly n terms. `polyPow(a, k, n)`: any a, $k \leq 10^{18}$. `polyDivMod(a, b)` returns (q, r). **Time:** $\mathcal{O}(n \log n)$
Usage: `polyExp(polyLog(a, n), n) == a`

```
vl polyInv(vl a, int n) { // a[0] != 0
  vl b = {modpow(a[0], mod - 2)};
  for (int m = 1; m < n; m *= 2) {
    vl f(a.begin(), a.begin()+min(sz(a), 2*m)), c = conv(f, b);
    c.resize(2 * m);
    for (ll& x : c) x = (mod - x) % mod;
    c[0] = (c[0] + 2) % mod;
    b = conv(b, c); b.resize(2 * m);
  }
  b.resize(n); return b;
}
vl polyDer(vl a) {
  rep(i,1,sz(a)) a[i-1] = a[i] * i % mod;
  if (!a.empty()) a.pop_back();
  return a;
}
vl polyInt(vl a) {
  a.insert(a.begin(), 0);
  vl inv(sz(a), 1);
  rep(i,2,sz(a)) inv[i] = (mod - mod/i*inv[mod%i] % mod) % mod;
  rep(i,1,sz(a)) a[i] = a[i] * inv[i] % mod;
  return a;
}
vl polyLog(vl a, int n) { // a[0] = 1
  vl c = conv(polyDer(a), polyInv(a, n));
  c.resize(n - 1); return polyInt(c);
}
vl polyExp(vl a, int n) { // a[0] = 0
  vl b = {1};
  for (int m = 1; m < n; m *= 2) {
    vl l = polyLog(b, 2 * m), f(2 * m);
    rep(i,0,2*m) f[i] = ((i<sz(a) ? a[i] : 0)-l[i]+mod) % mod;
    f[0] = (f[0] + 1) % mod;
    b = conv(b, f); b.resize(2 * m);
  }
  b.resize(n); return b;
}
vl polyPow(vl a, ll k, int n) { // a^k mod x^n, k >= 0
  int z = 0;
  while (z < sz(a) && !a[z]) z++;
  vl r(n);
  if (!k) { r[0] = n > 0; return r; }
  if (z == sz(a) || z > (n - 1) / k) return r; // z * k >= n
  int m = n - (int)(z * k);
  ll c = a[z], ci = modpow(c, mod - 2);
  vl b(a.begin() + z, a.end());
  for (ll& x : b) x = x * ci % mod;
  b = polyLog(b, m); // a^k = c^k x^(zk) exp(k log b)
  for (ll& x : b) x = x * (k % mod) % mod;
  b = polyExp(b, m);
  ll ck = modpow(c, k);
  rep(i,0,m) r[z * k + i] = b[i] * ck % mod;
  return r;
}
vl polySqrt(vl a, int n) { // a[0] = 1
  return polyPow(a, (mod + 1) / 2, n);
}
pair<vl, vl> polyDivMod(vl a, vl b) { // b.back() != 0
  int n = sz(a), m = sz(b);
  if (n < m) return {{}, a};
  vl ra(a.rbegin(), a.rend()), rb(b.rbegin(), b.rend());
  vl q = conv(ra, polyInv(rb, n - m + 1)), r = b;
  q.resize(n - m + 1), reverse(all(q));
  r = conv(b, q);
  rep(i,0,m-1) r[i] = (a[i] - r[i] + mod) % mod;
  r.resize(m - 1);
  return {q, r};
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

"ModPow.h"

c5d9f0, 51 lines

Description: Integers mod the global `mod` (prime $< 2^{30}$): arithmetic, comparisons, I/O, `fpow(a, p)`, `inverse(a)`; `(int)a` is the value. **Time:** $\mathcal{O}(\log p)$ for `fpow`
Usage: `mi a = 5, b = 7; cout << a / b << ' ' << fpow(a, 100); vmi v(n);`

```
struct mint { // mod: prime < 2^30
  int v;
  explicit operator int() const { return v; }
  mint() : v(0) {}
  mint(ll x) : v((int)(x % mod)) { if (v < 0) v += mod; }
  bool operator==(mint o) const { return v == o.v; }
  friend bool operator!=(mint a, mint b) { return !(a == b); }
  friend bool operator<(mint a, mint b) { return a.v < b.v; }
  friend istream& operator>>(istream& is, mint& a) {
    ll x; is >> x; a = mint(x); return is;
  }
  friend ostream& operator<<(ostream& os, mint a) {
    return os << (int)a;
  }
  mint& operator+=(mint o) {
    if ((v += o.v) >= mod) v -= mod;
    return *this;
  }
  mint& operator-=(mint o) {
    if ((v -= o.v) < 0) v += mod;
    return *this;
  }
  mint& operator*=(mint o) {
    v = (int)((ll)v * o.v % mod);
    return *this;
  }
  mint& operator/=(mint o) { return *this *= inverse(o); }
  friend mint fpow(mint a, ll p) { // p >= 0
    mint r = 1;
    for (; p; p /= 2, a *= a) if (p & 1) r *= a;
    return r;
  }
  friend mint inverse(mint a) {
    assert(a.v); return fpow(a, mod - 2);
  }
  mint operator-() const { return mint(-v); }
```

```
  mint& operator++() { return *this += 1; }
  mint& operator--() { return *this -= 1; }
  mint operator++(signed) {
    mint r = *this; ++*this; return r;
  }
  mint operator--(signed) {
    mint r = *this; --*this; return r;
  }
  friend mint operator+(mint a, mint b) { return a += b; }
  friend mint operator-(mint a, mint b) { return a -= b; }
  friend mint operator*(mint a, mint b) { return a *= b; }
  friend mint operator/(mint a, mint b) { return a /= b; }
};
using mi = mint;
using vmi = vector<mi>;
```

ModInverse.h

6f684f, 3 lines

Description: Inverses of $1..LIM-1$; LIM $\leq$ mod, mod prime.

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

dae77d, 8 lines

Description: The modulus shared by every header that needs one: set mod here. `modpow(b, e)` $= b^e \bmod mod$.

```
const int mod = 998244353; // prime; NTT needs c * 2^k + 1

ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

c040b8, 11 lines

Description: $a^x \equiv b \pmod m$ for any m. `modLog(a, 1, m)` is the order of a. **Time:** $\mathcal{O}\left(\sqrt{m}\right)$

```
ll modLog(ll a, ll b, ll m) { // min x > 0, or -1
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

5c5bc5, 16 lines

Description: `modsum(to, c, k, m)`: sum over $i < to$; `divsum`: the same with $\lfloor (ki+c)/m \rfloor$. **Time:** $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) { // sum of (ki+c) % m
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

bbbd8f, 11 lines

Description: $ab \bmod c$ and $a^b \bmod c$. **Time:** $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow`

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) { // a, b <= M <= 7.2e18
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

"ModPow.h" 19a793, 24 lines

Description: Tonelli-Shanks for prime p; $-x$ is the other root. **Time:** $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

```
ll sqrt(ll a, ll p) { // x^2 = a (p); asserts if none
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

PrimitiveRoot.h

"ModMulLL.h" 6d4b61, 15 lines

Description: Smallest primitive root. 998244353: 3, 10^9+7: 5. **Time:** $\mathcal{O}\left(\sqrt{p} + \log^2 p\right)$ expected

```
ull primitiveRoot(ull p) { // prime p <~ 1e12; else Factor
  if (p == 2) return 1;
  vector<ull> f;
  ull m = p - 1;
  for (ull d = 2; d * d <= m; d++) if (m % d == 0) {
    f.push_back(d);
    while (m % d == 0) m /= d;
  }
  if (m > 1) f.push_back(m);
  for (ull g = 2;; g++) {
    bool ok = 1;
    for (ull q : f) ok &= modpow(g, (p - 1) / q, p) != 1;
    if (ok) return g;
  }
}
```

5.2 Primality

FastEratosthenes.h

25a76b, 20 lines

Description: Primes below LIM; also fills `isPrime`. **Time:** LIM=1e9 $\approx$ 1.5s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

"ModMulLL.h" 60dcd1, 12 lines

Description: Deterministic for $n < 7 \cdot 10^{18}$. **Time:** 7 times the complexity of $a^b \mod c$.

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

Description: Prime factors with multiplicity, in any order (Pollard rho). **Time:** $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

5.3 Divisibility

euclid.h 33ba8f, 5 lines

Description: Returns $\gcd(a,b)$; if coprime, $x = a^{-1} \bmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y) { // ax + by = gcd
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

CRT.h "euclid.h" 04d93a, 7 lines

Description: $0 \le x <$ lcm if $|a| < m$, $|b| < n$; asserts if none. Needs $mn < 2^{62}$. **Time:** $\log(n)$

```
ll crt(ll a, ll m, ll b, ll n) { // x = a (m), x = b (n)
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

ExLucas.h "euclid.h" 22dc2e, 42 lines

Description: $\binom{n}{k} \bmod m$ for any $m < 2^{31}$ and $n \le mx$ (extended Lucas: per prime power, $n!$ without the factors p, then CRT). **Time:** $\mathcal{O}\left(\sqrt{m} + \omega(m) \cdot mx\right)$ build, $\mathcal{O}\left(\omega(m) \log n\right)$ per query.
Usage: `BinomMod C(m, maxn); C(n, k);`

```
struct BinomPP {
  ll p, q, c, a = 0; vector<ll> f;
  ll inv(ll b) { ll x, y; euclid(b, q, x, y); return (x % q + q
      ) % q; }
  BinomPP(ll p, ll q, ll mx, ll m) : p(p), q(q), f(min(q, mx +
      1), 1) {
    rep(i,1,sz(f)) f[i] = f[i-1] * (i % p ? i : 1) % q;
    c = m / q * inv(m / q % q) % m;
    for (ll t = q; t > 1; t /= p) a++;
  }
  ll nu(ll n) { ll e = 0; while (n /= p) e += n; return e; } //
       v_p(n!)
  ll fact(ll n) { // n! / p^nu(n) mod q
    ll r = 1;
    for (; n; n /= p) {
      if ((p != 2 || q <= 4) && n / q % 2) r = q - r;
      r = r * f[n % q] % q;
    }
    return r;
  }
  ll C(ll n, ll k) {
    ll e = nu(n) - nu(k) - nu(n - k), r;
    if (e >= a) return 0;
    for (r = fact(n) * inv(fact(k) * fact(n - k) % q) % q; e
        --;) r = r * p % q;
    return r;
  }
};

struct BinomMod {
  ll m; vector<BinomPP> v;
  BinomMod(ll m, ll mx) : m(m) {
    for (ll p = 2, x = m; x > 1; p++) {
      if (p * p > x) p = x;
      ll q = 1;
      while (x % p == 0) x /= p, q *= p;
      if (q > 1) v.emplace_back(p, q, mx, m);
    }
  }
  ll operator()(ll n, ll k) {
    if (k < 0 || k > n) return 0;
    ll r = 0;
    for (auto& b : v) r = (r + b.C(n, k) * b.c) % m;
    return r;
  }
};
```

5.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = gcd(a,b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x,y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

$ax + by = c$ is solvable iff $g = \gcd(a,b) \mid c$: take $(x_0, y_0) = (x_g, y_g) \cdot c/g$ from `euclid`, then $x = x_0 + \frac{b}{g}t$, $y = y_0 - \frac{a}{g}t$. For $a, b > 0$, $x \in [L, R]$ means $t \in [\lceil (L - x_0)g/b \rceil, \lfloor (R - x_0)g/b \rfloor]$ and $y \in [L', R']$ means $t \in [\lceil (y_0 - R')g/a \rceil, \lfloor (y_0 - L')g/a \rfloor]$: intersect the ranges (careful with floor division of negatives). $x + y$ changes by $(b - a)/g$ per step of t, so it is minimal at an end of the range.

phiFunction.h cf7d6d, 8 lines

Description: Sieve of Euler's ϕ below LIM. $\phi(n) = n \prod_{p \mid n} (1 - 1/p)$, $\sum_{d \mid n} \phi(d) = n$, $\sum_{k \le n, \gcd(k,n)=1} k = n\phi(n)/2$ $(n > 1)$. Euler: $a^{\phi(n)} \equiv 1 \pmod{n}$ if $\gcd(a,n) = 1$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.3.2 Garner's algorithm

CRT for pairwise coprime $m_0, \ldots, m_{k-1}$ (for two moduli see `crt`): write $x = x_0 + x_1 m_0 + x_2 m_0 m_1 + \cdots + x_{k-1} m_0 \cdots m_{k-2}$ with $0 \le x_i < m_i$. Then $x_i = \left(a_i - x_0 - x_1 m_0 - \cdots - x_{i-1} m_0 \cdots m_{i-2}\right)(m_0 \cdots m_{i-1})^{-1} \bmod m_i$, in $O(k^2)$ with the inverses $m_j^{-1} \bmod m_i$ precomputed:

```
for i: t = a[i];
  for j < i: t = (t-x[j])
    * inv(m[j], m[i]) % m[i];
  x[i] = t;
```

Then $x \bmod M$ for any M by Horner (e.g. a convolution result mod M from three NTT primes):

```
r = x[k-1]; for i = k-2..0:
  r = (r*m[i] + x[i]) % M;
```

5.3.3 Exponents

Generalized Euler: $a^b \equiv a^{(b \bmod \phi(m)) + \phi(m)} \pmod{m}$ for every a and every $b \ge \log_2 m$ (use it for a^{b^c} and power towers). Lifting the exponent: for an odd prime p with $p \mid x - y$ and $p \nmid xy$, $v_p(x^n - y^n) = v_p(x - y) + v_p(n)$; if n is odd and $p \mid x + y$, $v_p(x^n + y^n) = v_p(x + y) + v_p(n)$. For $p = 2$ and odd x, y: $v_2(x^n - y^n) = v_2(x - y)$ for odd n, and $v_2(x - y) + v_2(x + y) + v_2(n) - 1$ for even n (so $v_2(x - y) + v_2(n)$ if $4 \mid x - y$).

5.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \ \ b = k \cdot (2mn), \ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.5 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.6 Floor blocks

$\lfloor n/i \rfloor$ takes $O(\sqrt{n})$ distinct values; the blocks $[l, r]$ with equal $\lfloor n/i \rfloor$ are:

```
for (l = 1; l <= n; l = r + 1) {
  r = n / (n / l); ... }
```

$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/(ab) \rfloor$, and
$\sum_{i=1}^{n} \lfloor n/i \rfloor = 2 \sum_{i \le \lfloor \sqrt{n} \rfloor} \lfloor n/i \rfloor - \lfloor \sqrt{n} \rfloor^2$.

Lucy.h 328402, 20 lines

Description: $\sum_{p \le v} f(p)$ over primes, for all $v = \lfloor n/i \rfloor$; f completely multiplicative (`1`: count, `p`: sum), `g(v)` $= f(2) + \cdots + f(v)$ in closed form. The result type is `g`'s (`ull`: mod 2^{64}). **Time:** $\mathcal{O}\left(n^{3/4}\right)$, about 0.4 s for $n = 10^{11}$
Usage: `Lucy L(n, [](ll) { return 1LL; }, [](ll v) { return v - 1; }); // L(n) = pi(n)`

```
template<class F, class G> struct Lucy {
  typedef decltype(declval<G>()(0LL)) T;
  ll n, s; vector<T> lo, hi;
  Lucy(ll n, F f, G g) : n(n), s((ll)sqrtl((long double)n)),
      lo(s + 2), hi(s + 2) {
    while (s * s > n) s--;
    while ((s + 1) * (s + 1) <= n) s++;
    rep(i,1,s+1) lo[i] = g(i), hi[i] = g(n / i);
    for (ll p = 2; p <= s; p++) if (lo[p] != lo[p - 1]) {
      T fp = f(p), pre = lo[p - 1];
      for (ll i = 1; i <= s && n / i >= p * p; i++) {
        ll d = i * p;
        hi[i] -= fp * ((d <= s ? hi[d] : lo[n / d]) - pre);
      }
      for (ll v = s; v >= p * p; v--)
        lo[v] -= fp * (lo[v / p] - pre);
    }
  }
  T operator()(ll v) { return v <= s ? lo[v] : hi[n / v]; }
```

```
};
```

DuSieve.h a5bf54, 39 lines

Description: Prefix sums Φ (128-bit) and M of ϕ and μ at any $v = \lfloor n/i \rfloor$ or $v \le n^{2/3}$. Any f with $f * g = h$ and known prefix sums G, H: $S_f(v) = (H(v) - \sum_{d \ge 2} g(d) S_f(\lfloor v/d \rfloor))/g(1)$. **Time:** $\mathcal{O}\left(n^{2/3}\right)$, about 0.3 s for $n = 10^{10}$
Usage: `Du D(n); D.phi(n); D.mu(n); // also D.phi(n / 3)`

```
struct Du {
  typedef __int128 L;
  ll n, B; vector<ll> sp, sm, bm; vector<L> bp;
  Du(ll n) : n(n), B(max<ll>(10, (ll)cbrtl((long double)n) *
      (ll)cbrtl((long double)n))), sp(B + 1), sm(B + 1) {
    vi pr, phi(B + 1), mu(B + 1);
    phi[1] = mu[1] = 1;
    rep(i,2,B+1) {
      if (!phi[i])
        pr.push_back((int)i), phi[i] = (int)i - 1, mu[i] = -1;
      for (int p : pr) {
        if ((ll)i * p > B) break;
        if (i % p) {
          phi[i * p] = phi[i] * (p - 1), mu[i * p] = -mu[i];
        } else {
          phi[i * p] = phi[i] * p, mu[i * p] = 0;
          break;
        }
      }
    }
    rep(i,1,B+1) {
      sp[i] = sp[i-1] + phi[i], sm[i] = sm[i-1] + mu[i];
    }
    ll K = n / B;
    bp.resize(K + 1), bm.resize(K + 1);
    for (ll i = K; i >= 1; i--) {
      ll v = n / i, m = 1; L p = (L)v * (v + 1) / 2;
      for (ll l = 2, r; l <= v; l = r + 1) {
        r = v / (v / l);
        ll d = i * l; // v / l = n / d
        p -= (L)(r - l + 1) * (d <= K ? bp[d] : (L)sp[n / d]);
        m -= (r - l + 1) * (d <= K ? bm[d] : sm[n / d]);
      }
      bp[i] = p, bm[i] = m;
    }
  }
  L phi(ll v) { return v <= B ? (L)sp[v] : bp[n / v]; }
  ll mu(ll v) { return v <= B ? sm[v] : bm[n / v]; }
};
```

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h 558085, 18 lines

Description: Exact for $n \le 20$. Unknown size: `radixToPerm`. **Time:** $\mathcal{O}(n)$ and $\mathcal{O}(n^2)$
Usage: `intToPerm(0, 3) // {0, 1, 2}, intToPerm(5, 3) // {2, 1, 0}`

```
ll permToInt(vi& v) { // rank in lexicographic order
  ll use = 0, r = 0;
  int n = sz(v);
  rep(i,0,n) {
    int x = v[i];
    r = r * (n - i) + x
      - __builtin_popcountll(use & ((1LL << x) - 1));
    use |= 1LL << x;
  }
  return r;
}
vi intToPerm(ll r, int n) { // inverse: the r'th permutation
  vi c(n), s(n), v(n);
  iota(all(s), 0);
  for (int i = n; i--; r /= n - i) c[i] = (int)(r % (n - i));
  rep(i,0,n) v[i] = s[c[i]], s.erase(s.begin() + c[i]);
  return v;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

Corollaries: $p \nmid \binom{n}{m} \iff m_i \le n_i\ \forall i$. In particular, $\binom{n}{m}$ is odd $\iff$ `(m & n) == m`. The number of $m \in [0, n]$ with $p \nmid \binom{n}{m}$ is $\prod_i (n_i + 1)$ ($= 2^{\text{popcount}(n)}$ for $p = 2$).

6.2.3 Binomials

multinomial.h a0a312, 5 lines

Description: Exact while it fits in `ll`.

```
ll multinomial(vi& v) { // (sum k)! / prod k!
  ll c = 1, m = v.empty() ? 1 : v[0];
  rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
  return c;
}
```

6.2.4 Radix conversion

RadixConversion.h e542fb, 44 lines

Description: Bijective base conversion, most significant first: the k-th sequence (by length, then lexicographic) over $[0, a)$ goes to the k-th over $[0, b)$, $2 \le a, b < 2^{31}$: no length or leading zeros to store, about $|x| \log_b a$ digits. `radixToPerm` / `permToRadix`: the same onto permutations (size from the data). Fixed size n: `radixToPerm` of the base $2^{31} - 1$ digits of $r + 0! + \cdots + (n-1)!$. **Time:** $\mathcal{O}(|x| \cdot |y|)$

```
// y = y * m + c in bijective mixed radix: digit i (least
// significant first) is in [1, r(i)]
template<class F> void mulAdd(vector<ll>& y, ll m, ll c, F r) {
  rep(i,0,sz(y)) {
    ll t = y[i] * m + c, R = r(i);
    y[i] = (t - 1) % R + 1, c = (t - 1) / R;
  }
```

```
  while (c) {
    ll R = r(sz(y));
    y.push_back((c - 1) % R + 1), c = (c - 1) / R;
  }
}
vi radixconv(const vi& x, int a, int b) {
  vector<ll> y; // bijective base b, least significant first
  for (int d : x) mulAdd(y, a, d + 1, [&](int) { return b; });
  vi r;
  for (int i = sz(y); i--;) r.push_back((int)(y[i] - 1));
  return r;
}
vi radixToPerm(const vi& x, int a) {
  vector<ll> y; // bijective factorial base
  for (int d : x)
    mulAdd(y, a, d + 1, [](int i) { return i + 1; });
  int n = sz(y);
  vi s(n), p(n);
  iota(all(s), 0);
  rep(i,0,n) {
    int c = (int)y[n - 1 - i] - 1;
    p[i] = s[c], s.erase(s.begin() + c);
  }
  return p;
}
vi permToRadix(const vi& p, int a) {
  int n = sz(p);
  vector<ll> z; // bijective base a
  rep(k,0,n) {
    int c = 0;
    rep(j,k+1,n) c += p[j] < p[k];
    mulAdd(z, n - k, c + 1, [&](int) { return a; });
  }
  vi x;
  for (int i = sz(z); i--;) x.push_back((int)z[i] - 1);
  return x;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$
$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

6.4 Generating functions

6.4.1 Dictionary

OGF $A(x) = \sum_n a_n x^n$; EGF $\hat{A}(x) = \sum_n a_n x^n/n!$ (labeled objects), so $a_n = n![x^n]\hat{A}$.

a_n	OGF	EGF
$a_n + b_n$	$A + B$	$\hat{A} + \hat{B}$
$\sum_i a_i b_{n-i}$	AB	
$\sum_i \binom{n}{i} a_i b_{n-i}$		$\hat{A}\hat{B}$
a_{n-1}	xA	$\int \hat{A}$
a_{n+1}	$(A - a_0)/x$	$\hat{A}'$
na_n	xA'	$x\hat{A}'$
$\sum_{i \leq n} a_i$	$A/(1-x)$	

6.4.2 Common series

OGF: $\frac{1}{(1-x)^k} = \sum_n \binom{n+k-1}{k-1}x^n$, $\frac{x}{1-x-x^2} = \sum_n F_n x^n$, $\frac{1}{\sqrt{1-4x}} = \sum_n \binom{2n}{n}x^n$, Catalan $C = \frac{1-\sqrt{1-4x}}{2x} = 1 + xC^2$, partitions $\prod_{k\geq 1}\frac{1}{1-x^k}$.

EGF: $\frac{1}{1-x}$ (permutations), $\frac{e^{-x}}{1-x}$ (derangements), e^{e^x-1} (Bell), $e^{x+x^2/2}$ (involutions), $\frac{(e^x-1)^k}{k!}$ $(S(n,k))$, $\frac{(-\ln(1-x))^k}{k!}$ $(c(n,k))$, $\frac{x}{e^x-1}$ (Bernoulli).

6.4.3 Counting structures

Labeled, F the EGF of one kind of structure $(F(0) = 0)$: sequence $\frac{1}{1-F}$, set e^F, cycle $\ln\frac{1}{1-F}$, one marked element xF', pair $F \cdot G$. E.g. permutations are sets of cycles $(e^{\ln\frac{1}{1-x}} = \frac{1}{1-x})$, set partitions are sets of non-empty sets, rooted trees $T = xe^T$ give n^{n-1}; all cycle lengths in S: $\exp\left(\sum_{k\in S} x^k/k\right)$.

Unlabeled, F the OGF with f_k objects of size k: sequence $\frac{1}{1-F}$; multiset $\prod_k (1-x^k)^{-f_k} = \exp\sum_{j\geq 1} F(x^j)/j$ (Euler transform); set $\prod_k (1+x^k)^{f_k} = \exp\sum_{j\geq 1}(-1)^{j+1}F(x^j)/j$; rooted trees $T = x\exp\sum_{j\geq 1} T(x^j)/j$ $(1, 1, 2, 4, 9, 20, 48, \dots)$.

6.4.4 Computing

P/Q with $Q(0) = 1$: $a_n = \sum_i c_i a_{n-i}$ for $Q = 1 - \sum_i c_i x^i$ (see `BostanMori`, `BerlekampMassey`); partial fractions give closed forms.

Lagrange inversion: if $f = x\varphi(f)$ with $\varphi(0) \neq 0$, then $[x^n]f^k = \frac{k}{n}[x^{n-k}]\varphi^n$ and $[x^n]H(f) = \frac{1}{n}[x^{n-1}]H'\varphi^n$.

$b = e^F$ with $F(0) = 0$: $nb_n = \sum_{k=1}^{n} k f_k b_{n-k}$ in $O(n^2)$; in $O(n \log n)$ with `polyExp` (also `polyLog`, `polyInv`).

Store EGF coefficients as $a_n/n!$ and a product is a plain `conv`; converting to the sequence a_n multiplies coefficient n by $n!$.

Graph (7)

7.1 Network flow

PushRelabel.h

0ae1d4, 48 lines

Description: Max flow, fast in practice. Flow on an edge: its positive `f`. **Time:** $\mathcal{O}\left(V^2\sqrt{E}\right)$

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  }

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

MinCostMaxFlow.h

85db88, 82 lines

Description: Negative costs: call `setpi` first (no negative cycles). Flow on an edge: its positive `flow`. **Time:** $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#pragma push_macro("int")
#undef int
#include <bits/extc++.h>
#pragma pop_macro("int")

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
  }

  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });

    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        }
      }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
  }

  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
  }

  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
      rep(i,0,N) if (pi[i] != INF)
        for (edge& e : ed[i]) if (e.cap)
          if ((v = pi[i] + e.cost) < pi[e.to])
            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};
```

Dinic.h

d7f0f1, 42 lines

Description: Max flow. **Time:** $\mathcal{O}(VE\log U)$ with $U = \max|cap|$; $\mathcal{O}\left(\min(E^{1/2}, V^{2/3})E\right)$ if $U = 1$; $\mathcal{O}\left(\sqrt{V}E\right)$ for matching

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: After max flow, the s side of a min cut = vertices reachable from s through edges with positive residual capacity.

GlobalMinCut.h

8b0e19, 21 lines

Description: Stoer-Wagner on an adjacency matrix. **Time:** $\mathcal{O}(V^3)$

```
pair<int, vi> globalMinCut(vector<vi> mat) { // (weight, side)
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
```

```
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

"PushRelabel.h" 0418b3, 13 lines

Description: Gomory-Hu tree of an undirected graph: max flow u–v = min edge on the tree path. **Time:** $\mathcal{O}(V)$ Flow Computations

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

7.2 Matching

HopcroftKarp.h

731cfb, 20 lines

Description: g: neighbours of each left vertex; r: all -1, size of the right side; $r[i]$ = match of right vertex i or -1. **Time:** $\mathcal{O}\left(E\sqrt{V}\right)$

```
int hopcroftKarp(vector<vi>& g, vi& r) { // matching size
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

MinimumVertexCover.h

"HopcroftKarp.h" bfb654, 20 lines

Description: Bipartite. Size = max matching; complement = max independent set.

```
vi cover(vector<vi>& g, int n, int m) { // right j -> n+j
  vi match(m, -1);
  int res = hopcroftKarp(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
}
```

WeightedMatching.h

8f41c7, 24 lines

Description: Min-cost bipartite matching using every left node, $N \leq M$; returns (cost, match), L[i] $\to$ R[match[i]]. Max: negate the costs. **Time:** $\mathcal{O}\left(N^2M\right)$

```
template<class T>
pair<T, vi> weightedMatching(vector<vector<T>> &C) {
  int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
  vector<T> dist(m), pot(m);
  vi match(i), rev(m, -1), cols(m), prev(m);
  T d = 0, nd, cost = 0;
  while (i--) {
    rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      rep(j,s,m) {
        c = cols[j], nd = dist[c] - pot[c];
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]))
        dist[j] = nd, prev[j] = r;
    }
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c];
  }
  return {cost, match};
}
```

GeneralMatching.h

"../numerical/MatrixInverse-mod.h" cb1912, 40 lines

Description: Maximum matching (randomized, fails with probability N/mod). **Time:** $\mathcal{O}\left(N^3\right)$

```
vector<pii> generalMatching(int N, vector<pii>& ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) % mod;
      }
    }
  } while (matInv(A = mat) != M);
  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;
      } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

7.3 DFS algorithms

SCC.h

76b5c9, 24 lines

Description: Tarjan. **Time:** $\mathcal{O}(E+V)$

Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

BiconnectedComponents.h

c6b7c7, 32 lines

Description: Calls f with the edge ids of each biconnected component; an edge in none is a bridge. **Time:** $\mathcal{O}(E+V)$

Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps([&](const vi& edgelist) {...});

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
```

```
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}

template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

BlockCutTree.h

2684bd, 25 lines

Description: Nodes $0..n-1$ are the vertices, then one node per biconnected component (a bridge is one with 2 vertices), adjacent to its vertices; a forest. Cut vertex iff tree degree > 1; w separates u, v iff it is a cut vertex on their tree path. **Time:** $\mathcal{O}(N+M)$

```
vector<vi> blockCut(vector<vi>& g) {
  int n = sz(g), T = 0;
  vi num(n), low(n), st;
  vector<vi> t(n);
  auto dfs = [&](auto&& self, int v, int p) -> void {
    num[v] = low[v] = ++T; st.push_back(v);
    for (int u : g[v]) if (u != p) {
      if (num[u]) low[v] = min(low[v], num[u]);
      else {
        self(self, u, v);
        low[v] = min(low[v], low[u]);
        if (low[u] >= num[v]) {
          int b = sz(t); t.emplace_back();
          for (int x = -1; x != u;) {
            x = st.back(); st.pop_back();
            t[b].push_back(x); t[x].push_back(b);
          }
          t[b].push_back(v); t[v].push_back(b);
        }
      }
    }
  };
  rep(v,0,n) if (!num[v]) dfs(dfs, v, -1);
  return t;
}
```

2sat.h

5f9706, 56 lines

Description: Negation: ~x. **Time:** $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

780b64, 15 lines

Description: Euler path / cycle from src, undirected or directed. Lists of (dest, edge id); an undirected edge has one id both ways. Returns the nodes (src at both ends for a cycle), empty if none. Edge ids: also push .second.
Time: $\mathcal{O}(V+E)$

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

7.4 Heuristics

MaximalCliques.h

b0d5b1, 12 lines

Description: Calls f with every maximal clique. Symmetric bitset matrix, no self-edges. **Time:** $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

MaximumClique.h

e81d4b, 49 lines

Description: Symmetric bitset matrix, no self-edges. Max independent set: a max clique of the complement. **Time:** Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max<int>(sz(qmax)-sz(q)+1,1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
```

```
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

MaximumIndependentSet.h

Description: Max clique of the complement. Bipartite: complement of `MinimumVertexCover`.

7.5 Trees

BinaryLifting.h

bfce85, 25 lines

Description: Jump tables for k-th ancestors and LCA; the root's parent must be itself. **Time:** construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

LCA.h

"../data-structures/RMQ.h" 0f62fb, 21 lines

Description: Euler tour + RMQ; root 0, adjacency list (directed or undirected). **Time:** $\mathcal{O}(N \log N + Q)$

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

CompressTree.h

"LCA.h" 9775a0, 21 lines

Description: Virtual tree of a subset: the subset plus pairwise LCAs. Returns (parent index, original node); the root is first and is its own parent. **Time:** $\mathcal{O}(|S| \log |S|)$

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  }
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

HLD.h

"../data-structures/LazySegTreeBlock.h", "../data-structures/LazySegTree.h" 3d7ef1, 46 lines

Description: Performs HLD over a `LazySeg` (edit its block): updates take `L`, queries return `S`, values start as `S()`. Full adjacency list, root 0. **Time:** $\mathcal{O}\left((\log N)^2\right)$

```
template <bool VALS_EDGES> struct HLD { // VALS_EDGES: on edges
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;
  LazySeg tree;
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N),pos(N),tree(vector<S>(N)){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(all(adj[u]), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  template <class B> void process(int u, int v, B op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, L x) {
    process(u, v, [&](int l, int r) { tree.upd(l, r, x); });
  }
  S queryPath(int u, int v) {
    S res = e();
    process(u, v, [&](int l, int r) {
        res = op(res, tree.query(l, r));
    });
    return res;
  }
  S querySubtree(int v) { // modifySubtree is similar
    return tree.query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
  }
};
```

CentroidDecomposition.h

a9f00e, 19 lines

Description: `run(f)` calls `f(c)` per centroid c; a DFS from c skipping `dead` then visits exactly its component. **Time:** $\mathcal{O}(N \log N)$ plus the work done in `f`

Usage: `Centroid C(g); C.run([&](int c){ ... });`

```
struct Centroid { // cpar: centroid tree, root -1
  vector<vi> g; vi sub, cpar; vector<char> dead;
  Centroid(vector<vi> g) : g(g), sub(sz(g)), cpar(sz(g), -1),
      dead(sz(g)) {}
  int calc(int v, int p) {
    sub[v] = 1;
    for (int u : g[v]) if (u != p && !dead[u]) sub[v] += calc(u
        , v);
    return sub[v];
  }
  int find(int v, int p, int n) {
    for (int u : g[v]) if (u != p && !dead[u] && sub[u]*2 > n)
      return find(u, v, n);
    return v;
  }
  template<class F> void run(F&& f, int v = 0, int p = -1) {
    int c = find(v, -1, calc(v, -1));
    cpar[c] = p; f(c); dead[c] = 1;
    for (int u : g[c]) if (!dead[u]) run(f, u, c);
  }
};
```

DsuOnTree.h

933a26, 28 lines

Description: Small to large: `add(v)` / `rem(v)` put one vertex into / out of a global structure; `query(v)` runs when it holds exactly the subtree of v. **Time:** $\mathcal{O}(N \log N)$ calls to add/rem

Usage: `DsuOnTree T(g, root);`
`T.run([&](int v){...}, [&](int v){...}, [&](int v){...});`

```
struct DsuOnTree {
  vector<vi> g; vi sub, big, tin, tout, ord; int tim = 0, root;
  DsuOnTree(vector<vi> g, int r = 0) : g(g), sub(sz(g)),
    big(sz(g), -1), tin(sz(g)), tout(sz(g)), ord(sz(g)),
    root(r) { init(r, -1); }
  void init(int v, int p) {
    sub[v] = 1; ord[tin[v] = tim++] = v;
    for (int u : g[v]) if (u != p) {
      init(u, v); sub[v] += sub[u];
      if (big[v] < 0 || sub[u] > sub[big[v]]) big[v] = u;
    }
    tout[v] = tim;
  }
  template<class A, class R, class Q>
  void go(int v, int p, bool keep, A& add, R& rem, Q& query) {
    for (int u : g[v]) if (u != p && u != big[v])
```

```
      go(u, v, 0, add, rem, query);
    if (big[v] >= 0) go(big[v], v, 1, add, rem, query);
    add(v);
    for (int u : g[v]) if (u != p && u != big[v])
      rep(i,tin[u],tout[u]) add(ord[i]);
    query(v);
    if (!keep) rep(i,tin[v],tout[v]) rem(ord[i]);
  }
  template<class A, class R, class Q> void run(A add, R rem, Q
      query) {
    go(root, -1, 1, add, rem, query);
  }
};
```

LinkCutTree.h

0fb462, 90 lines

Description: Forest with `link`, `cut` (must stay a forest) and `connected`.
Time: All operations take amortized $\mathcal{O}(\log N)$.

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

DirectedMST.h

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

Description: Minimum arborescence rooted at r: (cost, parents), cost -1 if none. **Time:** $\mathcal{O}(E \log V)$

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

7.6 Math

7.6.1 Spanning trees, Euler circuits, disjoint paths

Laplacian $L = D - A$, where A_{ij} is the number of edges $i \to j$ (or $i - j$), D the degrees (loops ignored); weights multiply. Delete row and column r and take the determinant: undirected, this is the number of spanning trees (any r); for a digraph $D = D_{out}$ (edge $a \to b$: `L[a][a]++, L[a][b]--`) counts spanning trees with all edges directed *towards* r, and $D = D_{in}$ (`L[b][b]++, L[a][b]--`) those directed *away* from r.

BEST theorem: if every vertex has in-degree = out-degree and the graph is connected, the number of Euler circuits (cyclic sequences of edges) is $t_r \prod_v (\deg v - 1)!$, where t_r is the number of trees towards r (the same for every r); written as sequences starting at a given vertex s: times $\deg s$.

LGV lemma: in a DAG with sources $a_1, \ldots, a_k$, sinks $b_1, \ldots, b_k$ and M_{ij} the number (or weighted sum) of paths $a_i \to b_j$, $\det M = \sum \operatorname{sgn}(\sigma) \prod w$ over the systems of vertex-disjoint paths $a_i \to b_{\sigma(i)}$; if only $\sigma = \mathrm{id}$ can occur it counts them.

7.6.2 Flows with lower bounds

Edge $u \to v$ with bounds $[l, c]$: give it capacity $c - l$ and set $\text{ex}[v] \mathrel{+}= l$, $\text{ex}[u] \mathrel{-}= l$. A node bounded by $[A, B]$ is split into $v_{in} \to v_{out}$ with bounds $[A, B]$. Add $S^* \to v$ of capacity $\text{ex}[v]$ if $\text{ex}[v] > 0$, and $v \to T^*$ of capacity $-\text{ex}[v]$ if $\text{ex}[v] < 0$. A feasible flow exists iff the max flow $S^* \to T^*$ saturates all edges out of S^*. For an s–t flow also add $t \to s$ of capacity ∞; its flow F is the flow value. Then delete it: *max* flow is $F + \text{maxflow}(s \to t)$ and *min* flow is $F - \text{maxflow}(t \to s)$, both in the residual graph. Paths covering a DAG's vertices: bound every vertex by $[1, \infty]$, connect s to all sources and all sinks to t, take the min flow. Convex edge costs (e.g. x^2) in min-cost flow: use parallel unit edges with costs $1, 3, 5, \ldots$

7.6.3 Min cut models

Max-weight closure (choose items, each needs its prerequisites): $s \to i$ with capacity w_i if $w_i > 0$, $i \to t$ with $-w_i$ if $w_i < 0$, $i \to j$ with ∞ if i needs j; answer $= \sum_{w_i > 0} w_i -$ mincut (chosen items are on the s side). Bipartite graph: max matching = min vertex cover (König), max independent set = $V -$ matching. With weights, min weight vertex cover is the min cut of $s \to L$ (w), $L \to R$ (∞), $R \to t$ (w). DAG min path cover (vertex-disjoint) $= N -$ matching in the split graph (edge $u_{out} \to v_{in}$); paths may share vertices: use the transitive closure. Dilworth: max antichain = min chain cover (the latter with the transitive closure). Edge-disjoint paths: unit capacities; vertex-disjoint: split v into $v_{in} \to v_{out}$ of capacity 1. Two labels per item with pairwise penalties: s side / t side is the label; penalty edges between neighbours (needs submodular costs).

7.6.4 Segment tree graph

To add edges $v \to [l, r]$ cheaply, build a segment tree whose nodes are graph vertices, with edges parent $\to$ child (cost 0), and connect v to the $O(\log N)$ nodes covering $[l, r]$. For $[l, r] \to v$ use a second tree with the same leaves and edges child $\to$ parent. This gives $O(N + M \log N)$ edges for Dijkstra, SCC, topological sort or BFS.

7.6.5 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

47ec0a, 28 lines

Description: T: double or long long (avoid int).

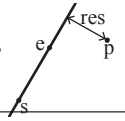
```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

"Point.h" 53630a, 4 lines

Description:

Signed distance from p to the line through a and b; a==b gives nan.



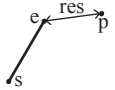

```
template<class P>
double lineDist(P a, P b, P p) { //> 0: left of a -> b
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

"Point.h" d5f39f, 6 lines

Description:

Distance from p to the segment se.



Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

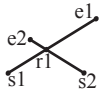
```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

"Point.h", "OnSegment.h" 9d57f2, 13 lines

Description:

One point, none, or the two endpoints of the common part. With Point<ll> the point is wrong if not integral. Products of 3 coordinates.



Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`
`if (sz(inter)==1)`
`cout << "segments intersect at " << inter[0] << endl;`

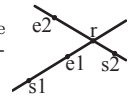
```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

lineIntersection.h

"Point.h" a01f81, 8 lines

Description:

{1, p} if unique, {0, (0,0)} if parallel, {-1, (0,0)} if the same line. With Point<ll> only integral points. Products of 3 coordinates.



Usage: `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1)`
`cout << "intersection point at " << res.second << endl;`

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

"Point.h" ca4665, 9 lines

Description: 1 / 0 / −1: p left of / on / right of $s \to e$; with eps, within distance eps counts as on.

Usage: `bool left = sideOf(p1,p2,q)==1;`

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

"Point.h" c597e8, 3 lines

Description: Doubles: use `segDist(s,e,p) <= eps` instead.

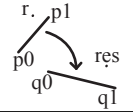
```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

"Point.h" 7871e9, 5 lines

Description:

Maps r by the similarity taking $p_0 \to q_0$, $p_1 \to q_1$.




```
typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h "Point.h" b5562d, 5 lines

Description: Projection of p on line ab (reflection if `refl`). Point<ll>: only integral results; products of 3 coordinates.

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h 0f0602, 35 lines

Description: Angles as int points plus full turns `t`, totally ordered; for rotational sweeps.
Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted`
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
`// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i`

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h "Point.h" 84d6d3, 11 lines

Description: False if none, else the two points in `out`.

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h "Point.h" b0153d, 13 lines

Description: External tangents (internal: negate r2; from a point: r2 = 0) as (point on c1, point on c2): none if one circle contains the other (internal: also if they overlap), 1 if they touch (`.first = .second`), else 2.

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

CircleLine.h "Point.h" e0cfba, 9 lines

Description: 0, 1 or 2 points.

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h "../../content/geometry/Point.h" 19add1, 19 lines

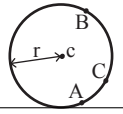
Description: Area of circle $\cap$ ccw polygon. **Time:** $\mathcal{O}(n)$

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

circumcircle.h "Point.h" 6cffc5, 9 lines

Description:

Radius and center of the circle through A, B and C.




```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h "circumcircle.h" 09dd0a, 17 lines

Description: Smallest enclosing circle (center, radius). **Time:** expected $\mathcal{O}(n)$

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

8.3 Polygons

InsidePolygon.h "Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

Description: `strict`: the boundary is outside. Products of coordinates (overflow). **Time:** $\mathcal{O}(n)$
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h "Point.h" f12300, 6 lines

Description: Twice the signed area.

```
template<class T>
T polygonArea2(vector<Point<T>>& v) { // 2x area, cw < 0
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

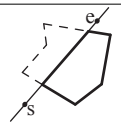
PolygonCenter.h "Point.h" 9706dc, 9 lines

Description: Center of mass. **Time:** $\mathcal{O}(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h "Point.h" d07181, 13 lines

Description:



Keeps the part to the right of $s \to e$.

Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

HalfPlane.h "lineIntersection.h" 77549e, 29 lines

Description: Half-plane (s, e) = left of $s \to e$. Returns the intersection in ccw order, empty if empty (or degenerate). Use doubles; add a big bounding box if it may be unbounded. **Time:** $\mathcal{O}(n \log n)$
Usage: `vector<pair<P,P>> h = {{s1, e1}, {s2, e2}, ...};`

```
template<class P> vector<P> halfPlanes(vector<pair<P, P>> h) {
  typedef pair<P, P> H;
  const double eps = 1e-9;
  auto dir = [](H& l) { return l.second - l.first; };
  auto out = [&](H& l, P p) { return dir(l).cross(p - l.first)
       < -eps; };
  auto inter = [](H& a, H& b) {
    return lineInter(a.first, a.second, b.first, b.second).
        second; };
  sort(all(h), [&](H a, H b) { return dir(a).angle() < dir(b).
      angle(); });
  vector<H> q(sz(h));
  int a = 0, b = 0;
  for (H& l : h) {
    for (; b - a > 1 && out(l, inter(q[b-1], q[b-2])); b--);
    for (; b - a > 1 && out(l, inter(q[a], q[a+1])); a++);
    if (b > a && fabs(dir(l).cross(dir(q[b-1]))) < eps) {
      if (dir(l).dot(dir(q[b-1])) < 0) return {};
      if (out(l, q[b-1].first)) b--; // same dir, l tighter
      else continue;
    }
    q[b++] = l;
  }
  for (; b - a > 2 && out(q[a], inter(q[b-1], q[b-2])); b--);
  for (; b - a > 2 && out(q[b-1], inter(q[a], q[a+1])); a++);
  if (b - a < 3) return {};
  rep(i,a,b) // bounded: turns left by < pi at each corner
    if (dir(q[i]).cross(dir(q[i+1 == b ? a : i+1])) < eps)
      return {};
  vector<P> r;
  rep(i,a,b) r.push_back(inter(q[i], q[i+1 == b ? a : i+1]));
  return r;
}
```

ConvexHull.h "Point.h" 310954, 13 lines

Description:



Counter-clockwise; collinear points are dropped.

Time: $\mathcal{O}(n \log n)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h "Point.h" c571b8, 12 lines

Description: Farthest pair of a ccw hull without collinear points. **Time:** $\mathcal{O}(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h "Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

Description: ccw hull without collinear points; `strict`: the boundary is outside. **Time:** $\mathcal{O}(\log N)$

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h "Point.h" 7cf45b, 39 lines

Description: Line ab vs a ccw convex polygon without collinear points: $(-1,-1)$ none, $(i,-1)$ touches corner i, (i,i) along side $(i,i+1)$, (i,j) crosses sides $(i,i+1)$ and $(j,j+1)$, in hit order. `extrVertex`: max projection. **Time:** $\mathcal{O}(\log n)$

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}
```

```
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

MinkowskiSum.h "Point.h" e9f890, 16 lines

Description: Of two ccw convex polygons (≥ 3 vertices, no 3 collinear), ccw. A and B intersect iff $0 \in A + (-B)$; their distance is the distance from 0 to it. **Time:** $\mathcal{O}(n+m)$

```
template<class P> vector<P> minkowski(vector<P> a, vector<P> b)
    {
  auto low = [](P p, P q) { return tie(p.y, p.x) < tie(q.y, q.x
      ); };
  rotate(a.begin(), min_element(all(a), low), a.end());
  rotate(b.begin(), min_element(all(b), low), b.end());
  int n = sz(a), m = sz(b);
  a.push_back(a[0]), a.push_back(a[1]);
  b.push_back(b[0]), b.push_back(b[1]);
  vector<P> r;
  for (int i = 0, j = 0; i < n || j < m;) {
    r.push_back(a[i] + b[j]);
    auto c = (a[i+1] - a[i]).cross(b[j+1] - b[j]);
    if (c >= 0 && i < n) i++;
    if (c <= 0 && j < m) j++;
  }
  return r;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h "Point.h" ac41a6, 17 lines

Description: Closest pair of ≥ 2 points. **Time:** $\mathcal{O}(n \log n)$

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
```

```
  return ret.second;
}
```

ManhattanMST.h

"Point.h" df6f59, 23 lines

Description: $\leq 4N$ edges (distance, i, j) that contain a Manhattan MST; run Kruskal on them. **Time:** $\mathcal{O}(N \log N)$

```
typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  }
  return edges;
}
```

RectangleUnion.h

ab84f0, 32 lines

Description: Area of the union; {x1, y1, x2, y2} is $[x_1, x_2) \times [y_1, y_2)$. **Time:** $\mathcal{O}(N \log N)$

```
typedef array<ll, 4> Rect;
ll rectUnion(vector<Rect> r) {
  vector<ll> ys;
  for (Rect& a : r) ys.push_back(a[1]), ys.push_back(a[3]);
  sort(all(ys)); ys.erase(unique(all(ys)), ys.end());
  int m = sz(ys) - 1;
  if (m < 1) return 0;
  vi c(4*m); vector<ll> cov(4*m);
  auto upd = [&](auto&& self, int o, int lo, int hi, int l, int
       r, int d) {
    if (r <= lo || hi <= l) return;
    if (l <= lo && hi <= r) c[o] += d;
    else {
      int mid = (lo + hi) / 2;
      self(self, 2*o, lo, mid, l, r, d); self(self, 2*o+1, mid,
           hi, l, r, d);
    }
    cov[o] = c[o] ? ys[hi] - ys[lo] : hi - lo == 1 ? 0 : cov[2*
        o] + cov[2*o+1];
  };
  vector<array<ll, 4>> ev; // x, -delta, y1, y2
  for (Rect& a : r) if (a[0] < a[2] && a[1] < a[3]) {
    ev.push_back({a[0], -1, a[1], a[3]});
    ev.push_back({a[2], 1, a[1], a[3]});
  }
  sort(all(ev));
  ll ans = 0, px = 0;
  for (auto& e : ev) {
    ans += cov[1] * (e[0] - px); px = e[0];
    int l = (int)(lower_bound(all(ys), e[2]) - ys.begin());
    int h = (int)(lower_bound(all(ys), e[3]) - ys.begin());
    upd(upd, 1, 0, m, l, h, (int)(-e[1]));
  }
  return ans;
}
```

kdTree.h

"Point.h" 69ff55, 63 lines

Description: 2D nearest neighbour: `nearest(p)` = (squared distance, point).

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(P a, P b) { return a.x < b.x; }
bool on_y(P a, P b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;

  T distance(P p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    }
  }
};

struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

  pair<T, P> search(Node *node, P p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(P p) {
    return search(root, p);
  }
};
```

Strings (9)

KMP.h

d4375c, 16 lines

Description: `pi`: prefix function; `match`: start positions of `pat` in `s`. **Time:** $\mathcal{O}(n)$

```
vi pi(const string& s) { // abacaba -> 0010123
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

Zfunc.h

ee09e2, 12 lines

Description: $z[i]$ = lcp of $s[i..]$ and s, $z[0] = 0$. **Time:** $\mathcal{O}(n)$

```
vi Z(const string& S) { // abacaba -> 0010301
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

e7ad79, 13 lines

Description: `p[1][i]`: half length (rounded down) of the longest odd palindrome centered at i; `p[0][i]`: of the even one centered before i. **Time:** $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

PalindromicTree.h

43784e, 24 lines

Description: Eertree: the other nodes are the distinct palindromic substrings; `link`: longest proper palindromic suffix; `last`: longest palindromic suffix of the prefix. `cnt[v]` after `count()`. **Time:** $\mathcal{O}(N)$ amortized, for a fixed alphabet

```
struct PalTree { // node 0: len -1, node 1: empty
  struct Nd {
    int len, link, cnt = 0; array<int, 26> nx;
    Nd(int len, int link) : len(len), link(link) {nx.fill(0);}
  };
  vector<Nd> t{Nd(-1, 0), Nd(0, 0)};
```

```
  string s; int last = 1;
  void add(char ch) {
    s += ch;
    int c = ch - 'a', i = sz(s) - 1;
    auto up = [&](int v) {
      while (i - t[v].len - 1 < 0 || s[i - t[v].len - 1] != ch)
            v = t[v].link;
      return v;
    };
    int cur = up(last);
    if (!t[cur].nx[c]) {
      int nw = sz(t), lk = t[cur].len < 0 ? 1 : t[up(t[cur].
          link)].nx[c];
      t.emplace_back(t[cur].len + 2, lk);
      t[cur].nx[c] = nw;
    }
    last = t[cur].nx[c]; t[last].cnt++;
  }
  void count() { for (int v = sz(t); v-- > 2;) t[t[v].link].cnt
        += t[v].cnt; }
};
```

MinRotation.h

67652e, 8 lines

Description: Start of the lexicographically smallest rotation. **Time:** $\mathcal{O}(N)$
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k==b || s[a+k]<s[b+k]) {b += max<int>(0,k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

SuffixArray.h

47bad6, 22 lines

Description: `sa` has size $n+1$ (`sa[0] = n`); `lcp[i] = lcp(sa[i], sa[i-1])`. No nul chars in the input. **Time:** $\mathcal{O}(n \log n)$

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j=0, p=0; p < n; j = max<int>(1, j*2), lim = p) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

SuffixAutomaton.h

3ac832, 30 lines

Description: State v: substrings with the same end positions, lengths (`len[link[v]], len[v]`]. Distinct substrings: $\sum_{v \neq 0}$ `len[v]-len[link[v]]`.
Time: $\mathcal{O}(N)$ states, $\mathcal{O}(N \cdot 26)$ memory, `count` in $\mathcal{O}(N \log N)$

```
struct SuffixAutomaton {
  struct S {
    int len, link = -1, cnt = 0; array<int, 26> nx;
    S(int len) : len(len) { nx.fill(-1); }
  };
  vector<S> t{S(0)}; int last = 0;
  void extend(char ch) {
    int c = ch - 'a', cur = sz(t), p = last;
    t.emplace_back(t[last].len + 1); t[cur].cnt = 1;
    for (; p >= 0 && t[p].nx[c] < 0; p = t[p].link) t[p].nx[c]
        = cur;
    if (p < 0) t[cur].link = 0;
    else {
      int q = t[p].nx[c];
      if (t[p].len + 1 == t[q].len) t[cur].link = q;
      else {
        int cl = sz(t);
        t.push_back(t[q]); t[cl].len = t[p].len + 1; t[cl].cnt
            = 0;
        for (; p >= 0 && t[p].nx[c] == q; p = t[p].link)
          t[p].nx[c] = cl;
        t[q].link = t[cur].link = cl;
      }
    }
    last = cur;
  }
  void count() { // cnt[v] = occurrences
    vi o(sz(t)); iota(all(o), 0);
    sort(all(o), [&](int a, int b) { return t[a].len > t[b].len
        ; });
    for (int v : o) if (t[v].link >= 0) t[t[v].link].cnt += t[v
        ].cnt;
  }
};
```

Hashing.h

2d2a67, 44 lines

Description: Hashing mod $2^{64}-1$; `hashInterval(a, b)` hashes $[a, b)$.

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

AhoCorasick.h

f35677, 66 lines

Description: Patterns non-empty (duplicates allowed); alphabet: `alpha, first`. `findAll(pat, word)`: all patterns starting at each position, shortest first. **Time:** construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) { // longest ending at i, -1
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
```

```
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

9.1 Suffix array and automaton facts

With `lcp[i] = lcp(sa[i], sa[i-1])`: the lcp of two suffixes with ranks $r_1 < r_2$ is $\min \texttt{lcp}[r_1+1 \dots r_2]$ (a range minimum query). Distinct substrings: $\sum_{i\geq 1}(n - \texttt{sa}[i] - \texttt{lcp}[i]) = n(n+1)/2 - \sum \texttt{lcp}$; suffix `sa[i]` adds the prefixes longer than `lcp[i]`, so the k'th distinct substring in lexicographic order is found by subtracting these counts along the suffix array. Longest common substring of s and u: build the automaton of s and run u through it, keeping a state v and a length (take a transition and add 1; if there is none, follow `link` and set the length to `len[v]`, until there is one); the answer is the largest length seen. Several strings: set `last = 0` before each string (all substrings are accepted and nothing else, and $\sum$ `len[v] - len[link[v]]` still counts the distinct substrings; the automaton may just have redundant states).

Various (10)

10.1 Intervals

IntervalMap.h — 7d8e78, 21 lines

Description: Partition of $[lo, hi)$ into intervals with values; `assign` merges equal neighbours; `each` visits the pieces of $[l, r)$ by reference. A set of disjoint intervals: `T = bool`. **Time:** amortized $\mathcal{O}(\log N)$ per assign, plus the pieces visited
Usage: `IntervalMap<int> m(0, n, 0); m.assign(l, r, v); m.each(l, r, [&](int a, int b, int& x){ ... }); m.get(i);`

```
template<class T> struct IntervalMap {
  map<int, T> m; int hi; // m[x] = value on [x, next key)
  typedef typename map<int, T>::iterator It;
  IntervalMap(int lo, int hi, T v) : hi(hi) {m[lo] = v; m[hi];}
  It split(int x) { // makes x a key
    It it = m.lower_bound(x);
    return it->first == x ? it :
      m.emplace_hint(it, x, prev(it)->second);
  }
  T get(int x) { return prev(m.upper_bound(x))->second; }
  template<class F> void each(int l, int r, F f) {
    It it = split(l), e = split(r);
    for (; it != e; ++it) f(it->first, next(it)->first, it->
        second);
  }
  void assign(int l, int r, T v) { // l < r
    It it = split(l), e = split(r);
    it->second = v; m.erase(next(it), e);
    if (e->first != hi && e->second == v) m.erase(e);
    if (it != m.begin() && prev(it)->second == v) m.erase(it);
  }
};
```

IntervalCover.h — 9e9d8d, 19 lines

Description: Fewest intervals covering G. Half-open (closed: add `|| R.empty()` at (A)). Empty on failure. **Time:** $\mathcal{O}(N \log N)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

10.2 Misc. algorithms

TernarySearch.h — 9155b4, 11 lines

Description: f strictly increasing, then non-increasing. Strict on the other side: <= at (A) and reverse (B). For the minimum, > at (A) and (B). **Time:** $\mathcal{O}(\log(b-a))$
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`

```
template<class F>
int ternSearch(int a, int b, F f) { // smallest argmax
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

10.3 Dynamic programming

DivideAndConquerDP.h — d38d2b, 18 lines

Description: $a[i] = \min_{lo(i)\leq k<hi(i)} f(i,k)$ for $i \in [L, R)$, when the smallest optimal k is non-decreasing in i. **Time:** $\mathcal{O}((N + (hi - lo)) \log N)$

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.3.1 Slope trick

To keep a convex piecewise-linear $f(x)$ with integer slopes, store the points where the slope changes by 1: left ones in a max-heap L, right ones in a min-heap R, and the minimum value m. Add $|x-a|$: push a to both heaps, fix the order of the tops, and add the difference to m if they cross. Add $\max(0, x-a)$: push a to L, move the top of L to R. Add $\max(0, a-x)$: push a to R, move the top of R to L. Prefix minimum: clear R. Shift by c: lazy offsets for L and R.

10.3.2 Sum over subsets (SOS DP)

Bits are numbered from 0 on the right. Let $S(x, i)$ be the set of submasks $y \subseteq x$ that equal x on all bits $\geq i$ (only the lowest i bits differ), e.g. $S(1011010_2, 4)$ is the pattern 101?0?0 with ? free: 1010000, 1010010, 1011000, 1011010. Then $S(x, 0) = \{x\}$, $S(x, n)$ is all submasks of x, and $S(x, i+1) = S(x, i)$ if bit i of x is 0, else $S(x, i) \cup S(x \oplus 2^i, i)$ (disjoint: bit i of y is 1 or 0). So $dp[x][i] = \sum_{y \in S(x,i)} a[y]$ has $dp[x][0] = a[x]$ and $dp[x][i+1] = dp[x][i] + [x_i = 1]\, dp[x \oplus 2^i][i]$. Layer i only reads entries with bit i clear, which it does not change, so one array is enough: $O(n2^n)$ time, $O(2^n)$ memory.

```
rep(i,0,n) rep(x,0,1<<n)
  if (x >> i & 1) f[x] += f[x ^ 1 << i];
```

Supersets: test `~x >> i & 1` and add `f[x ^ 1 << i]` (bit i is clear in x, so this is x with bit i set). Inverse (Mobius): same loops with $-=$. Any other operation works too (max, min, `|`), since the unions are disjoint. Pairs with $y \,\&\, z = 0$: z is a submask of $\sim y$.

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

10.6 Python

- Integers are unbounded, but `int(s)` and `str(x)` stop at 4300 digits with a `ValueError`: start with `sys.set_int_max_str_digits(0)` (a million digits take about 0.5 s).
- `math.isqrt(n)` is the exact integer square root; `int(math.sqrt(n))` is already wrong for $n = (10^{16}+1)^2 - 1$. Also `gcd`, `lcm`, `comb`, `perm`, `factorial`, `prod`.
- `pow(a, b, m)` is modular exponentiation and `pow(a, -1, m)` the modular inverse (`ValueError` if there is none). `//`, `%` and `divmod` round towards $-\infty$ (unlike C++). Also `x.bit_length()`, `x.bit_count()`, `int(s, base)`, `format(x, 'b')`.
- `decimal`: `getcontext().prec = 100` sets the digits kept in results. Build values from strings (`Decimal('0.1')`, not `Decimal(0.1)`). `.sqrt()`, `.ln()`, `.exp()`, `.log10()`, `.quantize(Decimal('0.01'), ROUND_HALF_UP)` (the default rounding is half-even: `2.665` gives `2.66`). Huge results such as $e^{3 \cdot 10^6}$ raise `Overflow` until `getcontext().Emax = MAX_EMAX`. `fractions.Fraction` is exact (`limit_denominator`).
- Input: `input = sys.stdin.readline` or `sys.stdin.buffer.read().split()`.
- Recursion: CPython 3.12 only needs `sys.setrecursionlimit(10**6)` for plain functions, but PyPy overflows its stack at depth 10^5 in the main thread. Run `main` in a thread: `threading.stack_size(2**29)`, `threading.Thread(target=main).start()` (2^{27} is enough for 10^5 levels, 2^{29} for 10^6). Recursion through `functools.lru_cache` fails on CPython 3.12 near depth 3300 whatever the limit: use a `dict` or iterate.
- `math.cbrt` (3.11+) does not exist on PyPy (Python 3.9); `bit_count` and `lcm` do.

Techniques (A)

techniques.txt 195 lines

```
Reformulate
  Contribution technique: sum what each element adds
  Fix one end, sweep the other: or fix the max / middle
  Reverse the process: deletions -> insertions, time backwards
  Invariants, monovariants: sum, parity, xor, mod k, sorted
  Differences / prefix sums: neighbour ops become simple there
  Manhattan <-> Chebyshev: (x + y, x - y) turns L1 into L-inf
  Bound the answer: often <= 2, <= log n or <= sqrt n: try each
  Binary search the answer: when checking a value is easy
  Complement counting: count the bad ones and subtract
  Swap state and value: dp[value] = min cost, values small
  Coloring argument: checkerboard or mod k colours, parity
  Pigeonhole: n + 1 prefix sums mod n: a block divisible by n
  Sqrt threshold: heavy (> sqrt) vs light items, two methods
  Few distinct sizes: sum of sizes = n: O(sqrt n) distinct
  Layered / state graph: node = (vertex, extra state), then BFS
  Functional graph: out-degree 1: rho shape, cycle + trees
  Bit by bit from the top: greedy on xor / and / or optimum
  Lexicographic minimum: fix positions in order, check feasible
  Balanced ternary: digits -1, 0, 1: balance-scale weights
Counting
  Linearity of expectation: sum of P(indicator), even dependent
  E[X] = sum P(X >= k): for X >= 0 integer, e.g. E[max]
  Reflection principle: bad paths = paths to the mirrored end
  Inclusion-exclusion: sum over subsets of violated rules
  Kummer: v_p(C(n, k)) = carries of k + (n-k) in base p
  Vandermonde: sum C(a, i) C(b, k-i) = C(a+b, k)
  Stars and bars: C(n + k - 1, k - 1) ways to split n into k
  Hall: match all of L iff |N(S)| >= |S| for all S
  Erdos-Szekeres: n^2 + 1 terms: monotone run of n + 1
  Coupon collector: n H_n expected draws
  Graph enumeration: connected labeled graphs: log of the EGF
  Entringer / zigzag: alternating permutations, boustrophedon
  Young tableaux: hook length formula; RSK and LIS
  Probability bounds: Markov, Chebyshev, Chernoff, union bound
Greedy
  Exchange argument: sort by the pairwise comparison a < b
  Regret greedy: heap of choices you can still undo
  Deadline scheduling: by deadline, heap; drop worst if late
  Huffman merging: merge the two smallest, cost = merges
  Matroid: greedy is optimal; two of them: intersection
Dynamic programming
  Aliens trick (WQS): exactly k, convex in k: penalty lambda
  Knuth optimization: opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
  Monotone queue DP: min over a sliding window of k
  Digit DP: count x <= N: (pos, tight, state)
  Broken profile DP: tilings of n x m, n small: bitmask row
  Rerooting: down pass, then up pass with prefix/suffix
  Tree knapsack: merging by subtree sizes is O(n^2) total
  DP over an automaton: state = (position, KMP / AC node)
  Dynamic DP: segment tree of (max,+) matrices, updates
  Steiner tree: dp[mask][v]: merge submasks, then Dijkstra
  Bounded knapsack: binary splitting, or monotone queue
  Bitonic tour: dp[i][j]: two paths ending at i and j
  Exchange DP order: iterate over the smallest dimension
  Expected value DP: backwards from the end; cycles: equations
  Garsia-Wachs: optimal alphabetic tree (merge cost)
Graphs
  Difference constraints: x_j - x_i <= w: edge i -> j, Bellman
  Residue Dijkstra: reachable sums: dist[r mod a_min]
  Min-plus matrix power: shortest walk with exactly k edges
  Kruskal reconstruction tree: path max edge, components <= w
  MST cycle property: heaviest cycle edge is never needed
  Boruvka: MST of an implicit graph (xor, geometry)
  Complement BFS: keep a set of unvisited, scan it
  Triangles in O(m sqrt m): orient edges low -> high degree
  Planar bounds: E <= 3V - 6; V - E + F = 2
  Grid is bipartite: (i + j) parity: matching / flow / cover
  Expected hitting times: linear system; on a tree, DP
  Tree isomorphism: canonical form or hash of sorted children
  Dominator tree: vertices every s -> v path passes
  Stable matching: Gale-Shapley proposals, O(n^2)
  0-1 BFS: deque: weight 0 front, weight 1 back
  Min mean cycle: Karp O(VE), or binary search + Bellman
  Shortest cycle: BFS from each vertex, O(VE)
  K-th shortest path: Eppstein; or A* with exact heuristic
  Euler tour flattening: subtree = a range of dfs times
  Long-path decomposition: depth DP in O(n) total
  Edge coloring: bipartite: max degree colours; else <= D + 1
  Vertex coloring: k colours: 2^n incl.-excl. over indep. sets
  Chordal graphs: perfect elimination order (max card. search)
  Max-weight general matching: weighted blossom, O(n^3)
  Min diameter spanning tree: absolute centre of the graph
  Dynamic graphs: extra book-keeping per change
  Lovasz toggle
Data structures
  Queue from two stacks: sliding min / gcd without undo
  Cartesian tree: recursion on the minimum; O(n) with stack
  Nearest smaller values: monotone stack; histogram rectangles
  Segment tree merging: merge dynamic trees small to large
  Rebuild every sqrt(Q): buffer updates, recompute periodically
  Binary trie: max xor pair, count xor <= k
  Zobrist / xor hashing: random 64-bit per item: set equality
  Merge sort tree / wavelet: k-th or count in a range, static
  Small to large: each item moves O(log n) times
Advanced data structures
  Splay tree: amortized log; accessed node goes to the root
  Balanced BSTs: AVL, red-black, AA, SBT, scapegoat
  WBLT: weight-balanced leafy tree: merge / split
  Skip list: random levels, expected O(log n)
  Pairing / leftist heap: mergeable heaps (leftist: rank)
  Persistent heap: leftist heap with path copying (k-th path)
  Persistent balanced tree: path-copied treap (persistent rope)
  Persistent block array: sqrt blocks, copy on write
  Finger tree: O(1) at both ends, log split / concat
  Disjoint sparse table: (cat tree) O(1) query, any assoc. op
  Sqrt tree: O(1) range query, O(n log log n) build
  Kinetic segment tree: max of a_i + t b_i as t only increases
  Euler tour tree: dynamic forest with subtree aggregates
  Top tree: dynamic trees, path and subtree ops
  Global balanced binary tree: HLD paths as weighted BSTs
  Block list: list of sqrt-size blocks, split / merge
  Tree blocking: cut a tree into O(sqrt n) clusters
  Tree in tree: segtree of treaps / BIT of value segtrees
  PQ tree: orders with consecutive-ones constraints
  Quadtree: 2D subdivision (usually kd-tree is better)
Offline
  CDQ divide and conquer: left half updates answer right half
  Parallel binary search: all queries binary search together
  Sort queries by right end: last occurrence: distinct counts
  D&C on the array: count pairs that cross the middle
  Discrete 2nd derivatives: add arithmetic progressions in O(1)
  Mo with updates: time as 3rd axis, blocks n^(2/3)
  Rollback Mo: only insertions: roll back to block start
  2D Mo: Mo over two dimensions / grids
  Secondary offline Mo: answer the pointer moves offline again
  Mo with bitset: bitset as the state of the window
  Interesting points in N log N
Number theory
  Frobenius number: ab - a - b largest non-sum, gcd = 1
  Stern-Brocot: best rational approximation, binary search
  gcd of powers: gcd(a^m-1, a^n-1) = a^gcd(m,n)-1; same for F
  Wilson: (p - 1)! = -1 mod p
  Sum of two squares: p = 1 mod 4 primes; Gaussian integers
  Vieta jumping: (x, y) -> (k y - x, y), descent
  Hensel lifting: root mod p -> mod p^k
  Pell equation: x^2 - d y^2 = 1 from continued fractions
  Min_25 sieve: prefix sums of multiplicative f
  Powerful number sieve: f = g * h, h only on powerful numbers
  Zhou's sieve: multiplicative prefix sums (like Min_25)
  k-th roots mod p: via primitive root + discrete log
  Quadratic fields: arithmetic in Z[sqrt d] (closed forms)
  Quadratic reciprocity: Legendre symbols of odd primes p, q
Polynomials and algebra
  Multipoint evaluation: subproduct tree, O(n log^2 n)
  Polynomial shift: p(x + c) with one convolution
  Chirp Z-transform: evaluate at a geometric sequence
  Composition: f(g(x)); compositional inverse
  Lagrange inversion: g = f^-1: [x^n] g = [x^(n-1)] (x/f)^n / n
  Characteristic polynomial: Hessenberg O(n^3); Cayley-Hamilton
  Diagonalization / Jordan: A^k from eigenvalues
  Linear programming: simplex; duality; zero-sum games
  Schreier-Sims: permutation group order and membership
Geometry
  Inversion: circles through the centre become lines
  Point-line duality: (a, b) <-> y = ax - b, hull <-> envelope
  Golden section search: unimodal real f, 1 eval per step
  Area by Green: union of circles / arcs: sum over arcs
  Segment intersections: Bentley-Ottmann, O((n + k) log n)
  Triangulation: ear clipping; Delaunay
  3D basics: cross products, planes, hull
Strings
  Period from KMP: p = n - pi[n-1]; full repeat iff p | n
  Next-occurrence table: nxt[i][c]: subsequence checks / counts
  Lyndon / Duval: factorization, min rotation, O(n)
  Hash + binary search: lcp / comparisons of substrings
  Main-Lorentz: all tandem repeats in O(n log n)
  Generalized SAM: one automaton for many strings
  SA-IS: linear-time suffix array
  Suffix balanced tree: online suffix order, add at the front
  Boyer-Moore: bad-character skips, sublinear on average
Games
  Sprague-Grundy: g = mex of options; sum: xor
  Nim: first player wins iff xor != 0
  Games with cycles: retrograde BFS from terminal states
  Token on a graph: first wins iff start in every max matching
  Mirror strategy: copy the opponent symmetrically
  Staircase nim: only odd steps matter
  Partizan games: surreal numbers, Hackenbush
  Alpha-beta pruning: minimax with cut-offs
Randomization and search
  Meet in the middle: n ~ 40: two halves, sort and combine
  Bitset speedup: n^3 / 64, subset sums, reachability
  Random sampling: property of >= half: try ~30 random picks
  Random shuffle: expected bounds (incremental, treap order)
  Simulated annealing: hill climbing that accepts worse moves
  IDA* / A*: admissible heuristic, iterative deepening
  Exact cover: dancing links (sudoku, pentomino)
Misc
  Fractional programming: max sum a / sum b: sum(a - x b) >= 0?
  Largest empty rectangle: histogram per row + monotone stack
  Josephus: J(n) = (J(n - 1) + k) mod n
  Majority vote: Boyer-Moore counter; mergeable in a segtree
  Gray code: i ^ (i >> 1): neighbours differ in 1 bit
  Kahan summation: compensated sum for floating error
```